

Clustering Based Minimum Spanning Tree Algorithm

Sakshi Saxena

sakshisaxena04@gmail.com

Priyanka Verma

priyanka7lko@gmail.com

Dharmveer Singh Rajpoot

dharmveer.rajpoot@jiit.ac.in

Computer Science and Engineering
Jaypee Institute of Information Technology
Noida, India

Abstract—Data mining is a technique used to process information from a big dataset and converting it into a reasonable form for supplementary use. Clustering is a mining technique used in data mining. The goal of clustering is to discover the groupings of a set of points, patterns or objects. Minimum Spanning Tree (MST) based clustering algorithms are successfully used for detecting clusters. In this paper we have focused on minimizing the time complexity for constructing MST by using clustering. The proposed algorithm tries to minimize the time complexity by constructing a MST in two stages. In divide stage, the given dataset is divided in various clusters. In the conquer stage, for every cluster, local MSTs are created and then these MSTs are combined to obtain the final MST by using Midpoint MST algorithm. Experimental results show that the proposed MST algorithm is computationally efficient.

Keywords—clustering; minimum spanning tree; k-means; data mining

I. INTRODUCTION

In clustering we divide the data into groups of similar objects. Each group or cluster can have objects similar or dissimilar to the objects in other groups [6]. Clustering algorithm based on graph theory is widely used in cluster analysis. They are efficient in solving a wide range of problem domains. This clustering algorithm is realized on the graph, where the data point is considered as a node and the edge represents the relationship among data points [4]. MST based clustering algorithms are widely used graph based clustering technique. This clustering technique can identify clusters independent of their shape and size [3]. A MST is a spanning tree of a weighted undirected graph having the minimum sum of weights.

Previous clustering based Minimum Spanning Tree algorithms works on divide and conquer scheme. In divide stage, the dataset is divided into various clusters using K means algorithm. In the conquer stage, these algorithms create local MSTs for every cluster and then combine these local MSTs to obtain the final MST with various approaches. These approaches have theoretical time complexity of $O(n^{3/2})$.

MST based clustering algorithms are successfully used to detect clusters of heterogeneous nature. Application of this algorithm is image colour clustering in web image analysis [7]. By using minimum spanning tree based clustering algorithms in web analyzing, thousands of colours are reduced

to a representatively few colours that clearly differentiate objects of interests in an image. Other applications of clustering involve pattern recognition, grouping, multi-objective decision making using machine-learning and operations like data mining, document retrieval, data image partitioning, and pattern classification [5].

In this work, we have proposed an algorithm to decrease the time complexity and increase the computational efficiency of the algorithm for constructing MST for a set of n points. The novelty of this work is that we have used Midpoint MST algorithm for joining local MSTs. In this approach the two local clusters are joined on the basis of their Euclidean distance from midpoint of the edge joining the centroids of clusters. This simplifies the process of finding edges between clusters and reduces the time complexity of this step.

The research is organized as follows: Section II: details and background of the previous work. Section III: presents the proposed algorithm. Section IV: presents the experimental results and evaluation of the algorithm. Section V: concludes and highlights the future work.

II. PREVIOUS WORK

A. Fast Minimum Spanning Tree algorithm based on k means (FMST)

C. Zhong et al. proposed a FMST algorithm based on k means [1]. It was divided into two stages. In the first stage the dataset is portioned into $\sqrt{n}$ clusters using k means. Then $\sqrt{n}$ MST are obtained by applying exact MST algorithm to each cluster, which are connected using an approximate MST. The second stage is called refinement stage, where the process is repeated for $\sqrt{n}-1$ clusters. Finally the MST is generated by combining the two approximate MSTs. This algorithm has $O(n^{3/2})$ time complexity.

B. MST based clustering on K means using Local Neighbourhood Graph (KMGClust)

R. Jothi et al. proposed a fast minimum spanning tree based clustering algorithms on local neighborhood graph [2]. In MST Based Clustering on K means graph the dataset X is partitioned into clusters by using k -means algorithm. Local Neighborhood graph is constructed and intra partition all pair edges is computed. For each pair of neighboring partitions inter partition edges is computed based on Centroid Based Rule. Then Kruskal's algorithm is applied on the local

neighborhood graph to obtain the Minimum Spanning Tree. This algorithm has $O(n^{3/2})$ time complexity.

C. MST based clustering on Bi means using Local Neighbourhood Graph (KMGClust)

R. Jothi et al. proposed a fast minimum spanning tree based clustering algorithms on local neighborhood graph [2]. In MST Based clustering on Bi Means Graph, the Binary partitioned tree of Dataset X is obtained. Inter partition edges and intra partition edges are identified on the set of partitions S . Then Kruskal's algorithm is applied on the Local Neighborhood graph to obtain the Minimum Spanning Tree.

III. PROPOSEDMETHOD

Let $X=\{x_1, x_2, x_3, x_4, \dots\}$ be a dataset of n dimensional points, the proposed algorithm finds an efficient minimum spanning tree for the provided dataset. Proposed algorithm works on divide and conquer approach. In divide step, the given dataset is divided in various clusters using K-Means algorithm. In the conquer step, for every cluster, local MSTs are created and then these MSTs are combined to obtain the final MST. The two local clusters are joined by using Midpoint MST algorithm. In this the points are selected on the basis of their Euclidean distance from midpoint of the edge joining the two centroids and then an edge is drawn between these two points.

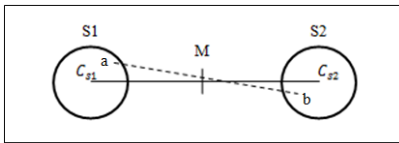


Fig. 1. Joining the points a and b nearest to the midpoint of the line joining the centroids of the clusters.

Let S_1 and S_2 be two clusters as shown in Figure 1. Let C_{s1} and C_{s2} be the centroids of cluster S_1 and S_2 . Let M be the midpoint of the line joining C_{s1} and C_{s2} . Let a and b are the points of S_1 and S_2 nearest to midpoint M , then a and b can efficiently join clusters S_1 and S_2 in order to form global MST.

A. Proposed Algorithm

The proposed algorithm is divided into three phases.

Input: Dataset of size n (n points in the dataset)

Output: Global MST

1) *Clustering*: To make the problem simple and efficient, the dataset is divided into various clusters by using k -means algorithm. This process is performed in step of line 2 of phase 1. For better result, clusters are formed from the dataset of n points. Figure 2 shows the formation of clusters.

2) *Creating local MSTs within the clusters*: Each cluster is processed individually to prepare local MST. The local MST can be created by using any minimum spanning tree

algorithm like Kruskal's, Prim's etc. Steps from line 1 to 5 of phase 2 represents the formation of local MSTs. Figure 3 shows the formation of local MSTs in each cluster.

3) *Combining local MSTs with centroids midpoints to obtain final MST*: Steps in phase 3 represents the formation of global MST using Midpoint MST Algorithm. The problem of joining the local MSTs efficiently to get final MST is done by joining the nearest Points to Clusters centroids midpoints. Figure 4 depicts the formation of final MST.

Phase1 : Clustering
Input: Dataset of size n Output: $\sqrt{n}$ clusters and set of centroids 'centroidList'
1. Calculate square root of n. 2. Apply k means algorithm with $k=\sqrt{n}$ 3. Save centroids in centroidList 4. Return centroidList

Phase 2 : Local MST Formation
Input: $\sqrt{n}$ clusters Output: Local MSTs of all clusters
1. For $i=1$ to $\sqrt{n}$ //Apply Kruskal's algorithm on each cluster 2. For each point p in cluster I, add edges with all other points of the cluster. 3. Sort the edges of G in ascending order. 4. For each edge (u,v) from the sorted list, if u and v belong to different sets 5. Add (u,v) to T 6. Get together u and v in one set

Phase 3: Formation of Global MST
Input: Set of Centroids Output: Global MST
1. For each two different points (u,v) in set of centroids calculate midpoint M 2. Find the nearest point of cluster u from midpoint M 3. Find the nearest point of cluster v from midpoint M 4. Calculate Euclidean distance between u and v 5. Add edge to set T 6. Apply Kruskal's algorithm on above set of edges.

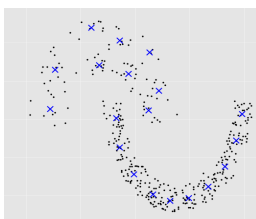


Fig. 2. Formation of clusters ('X' represent the centroid of each cluster)

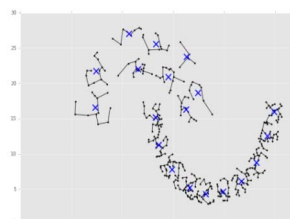


Fig. 3. Local MSTs within clusters

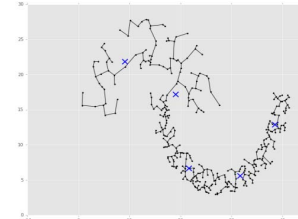


Fig. 4. Final MST

IV. EXPERIMENTAL RESULT

In this section, experimental results are presented. The results illustrate the efficiency, reliability and accuracy of the proposed Midpoint MST Algorithm. The efficiency is tested on the Jain dataset and t4.8k dataset. Here for making MST we have used Kruskal's algorithm. Other MST algorithms like Prim's algorithm can also be used. In Kruskal's algorithm, all the edges need to be sorted into non-decreasing order, thus it is difficult to apply on the large datasets.

Experiments were run on the PC with following specifications:

- Intel core i5 CPU
- 8GB RAM
- Windows 7 Ultimate

The algorithm for testing running time is implemented in Python 3.6 on Spyder 3 IDE.

The experiments are performed on two datasets with sizes and dimensions as mentioned in Table 1.

Table I Description of datasets

Data Set Name	Data Size	Dimension	Reference
t4.8k	8000	2	[8]
Jain	373	2	[9]

A. Results on Data Set t4.8k

- 1) *Clustering*: Figure 5 depicts the points in the dataset.
- 2) *Formation of Local MSTs in each cluster*: Figure 6 represents the local MSTs formed in the clusters.
- 3) *Formation of Final MST in t4.8k cluster*: Figure 7 depicts the formation of final MST in t4.8k cluster.

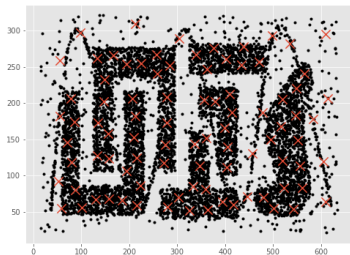


Fig. 5. 89 Clusters in t4.8k dataset

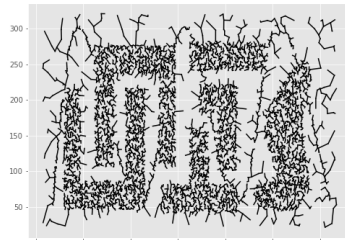


Fig. 6. Formation of Local MSTs in each cluster

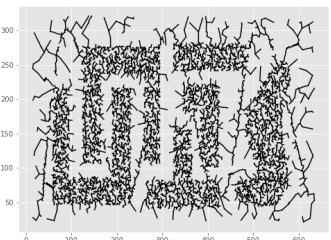


Fig. 7. MST of t4.8k cluster

V. CONCLUSION AND FUTURE SCOPE

In this paper, we have proposed a clustering based minimum spanning tree algorithm with a divide and conquer approach. In divide step, the dataset is divided in various clusters using k-means algorithm. In the conquer step for every cluster, local MSTs are created and then these MSTs are combined to obtain the final MST. In this work we have used Midpoint MST algorithm for combining local MSTs. Theoretical analysis indicates that the proposed algorithm has less time complexity. In the last step, the complexity of joining two clusters is reduced to $2\sqrt{n}$ from $O(n)$ during creation of complete graph to obtain global MST. Experiments indicate that the cost of the MST formed by our proposed algorithm is less than the cost of the MST formed by other MST based clustering algorithms.

As a future work, we will carry out an extensive analysis of our proposed algorithm on various dimensions of datasets. In future we can increase the efficiency of the proposed algorithm by calculating the points nearest to the midpoint by finding whether the clusters are aligned horizontally or vertically. In this approach, two points can be identified nearest to the midpoints aligned vertically or horizontally. Construction of final MST can be done by forming an edge between two points having minimum distance.

VI. REFERENCES

- [1] Zhong, Caiming, Mikko Malinen, Duoqian Miao, and Pasi Fränti. "A fast minimum spanning tree algorithm based on K-means." *Information Sciences* 295 (2015): 1-17.
- [2] Jothi, R., Sraban Kumar Mohanty, and Aparajita Ojha. "Fast minimum spanning tree based clustering algorithms on local neighborhood graph." In *International Workshop on Graph-Based Representations in Pattern Recognition*, pp. 292-301. Springer International Publishing, 2015.
- [3] Wang, Xiaochun, Xia Li Wang, Cong Chen, and D. Mitchell Wilkes. "Enhancing minimum spanning tree-based clustering by removing density-based outliers." *Digital Signal Processing* 23, no. 5 (2013): 1523-1538.
- [4] Xu, Dongkuan, and Yingjie Tian. "A comprehensive survey of clustering algorithms." *Annals of Data Science* 2, no. 2 (2015): 165-193.
- [5] Jain, Anil, and Martin Law. "Data clustering: A user's dilemma." *Pattern Recognition and Machine Intelligence* (2005): 1-10.
- [6] Berkhin, Pavel. "A survey of clustering data mining techniques." In *Grouping multidimensional data*, pp. 25-71. Springer Berlin Heidelberg, 2006.
- [7] Grygorash, Oleksandr, Yan Zhou, and Zach Jorgensen. "Minimum spanning tree based clustering algorithms." In *Tools with Artificial Intelligence, 2006. ICTAI'06. 18th IEEE International Conference on*, pp. 73-81. IEEE, 2006.
- [8] Karypis, George, Eui-Hong Han, and Vipin Kumar. "Chameleon: Hierarchical clustering using dynamic modeling." *Computer* 32, no. 8 (1999): 68-75.
- [9] Jain, Anil, and Martin Law. "Data clustering: A user's dilemma." *Pattern Recognition and Machine Intelligence* (2005): 1-10.